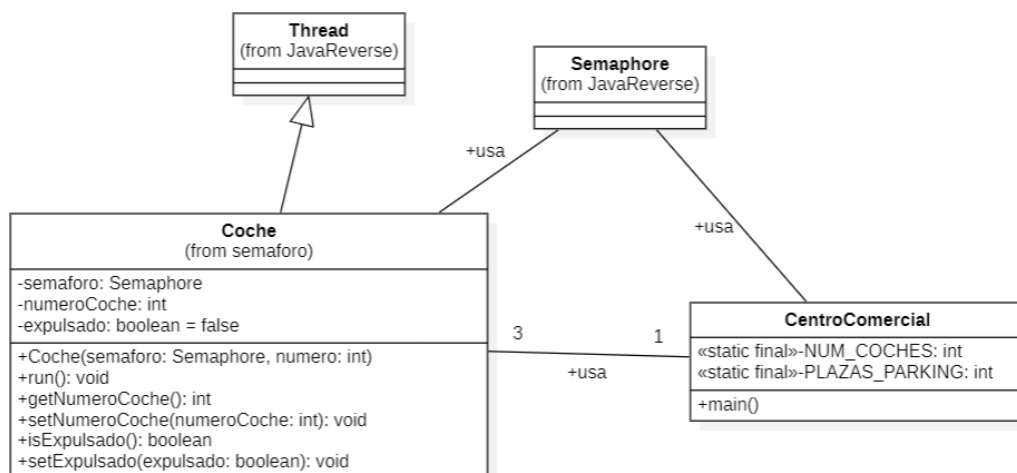


Ejercicio de clase: Semáforos y parking



Este ejercicio busca simular un sistema de estacionamiento utilizando hilos y semáforos en Java, permitiendo la gestión de coches en el parking y la posibilidad de expulsarlos por orden.



Clase Coche, extiende de Thread:

Tiene 2 atributos privados:

- Semáforo (clase *Semaphore*:
<https://docs.oracle.com/en/java/javase/16/docs/api/java.base/java/util/concurrent/Semaphore.html>)
- Numero de coche (entero)

Implementar el método “run”, con la siguiente lógica:

Dentro de un bucle while(true):

- Nada más empezar, hacer que el coche “de vueltas” por la ciudad, durmiendo un número aleatorio de milisegundos, de 1000 a 5000 millis
- Al despertar del sueño, decir que va a intentar entrar al parking
- Sacar por pantalla **cuántos coches hay en la cola** (del semáforo) y **el número de plazas libres** (“tickets” disponibles del semáforo)
- Sólo si el número de coches en la cola es menor o igual a 5,
 - Intentamos entrar al parking (es decir, **adquirimos** el semáforo)
 - Nos quedamos “dentro” un número de milisegundos aleatorio, entre 1000 y 5000, diciéndolo por pantalla y mostrando cuántos tickets quedan libres.
 - Nos “vamos” del parking (es decir, **soltamos** el semáforo)
- En caso contrario (es decir, si el número de coches en la cola es > 5)
 - Imprime en pantalla algo como “Hay mucha cola en el parking, mejor damos otra vuelta”.

Clase *CentroComercial*, tiene el método *main*

Declara 2 variables estáticas final:

- Número de plazas del parking
- Número de coches que existirán

Crearé el semáforo (el “parking”) con el número de plazas que tendrá.

Hará un bucle de 1 al número de coches total, haciendo:

- Una instancia nueva de la clase *Coche*, pasándole el semáforo y el número de coche
- Le asignará una prioridad igual a su número de coche
- Arrancará el hilo

PUNTOS IMPORTANTES:

- Haced un try/catch de la *InterruptedException* en la clase *Coche*.
- En la clase *coche*, **cada acción del coche tiene que tener una traza por pantalla**, para que sepamos qué es lo que hace el programa.
- Las trazas por pantalla que haga la clase *coche* deberán salir SIEMPRE con el número de coche delante. Por ejemplo: “**El coche 2** sale del parking” o “**El coche 5** recorre la ciudad 4500 ms”.
- El programa no terminará nunca, así que no consideréis eso como algo incorrecto. Lo podréis parar a voluntad.

CONSEJOS

- Utilizad el API que os he puesto arriba (para la clase *Semaphore*) para identificar los métodos que usaremos del semáforo (salen en negrita en el texto)

Extensión al ejercicio

Vamos a intentar expulsar a los coches.

Para ello, en la clase Coche:

- Incluir un atributo booleano “expulsado” dentro de la clase Coche, inicializado a false, que tenga un get y set. Añadir al bucle while(true) la condición de que sea **true && “no expulsado”**
- Cuando termine el bucle, decir **“EL COCHE ” + numeroCoche + “ HA SIDO EXPULSADO. FIN DE SU VIDA”**
- Poner en el try catch de InterruptedException un mensaje (El coche <numeroCoche> ha sido despertado” y además liberará el semáforo. Poned un mensaje también para que sepamos que se ha despertado, en mayúsculas.

Para ello, en la clase CentroComercial:

- Cada coche creado en la clase CentroComercial se mete en una lista de coches (List<Coche>)
- Después de inicializar todos, hacemos un bucle que tiene esta condición: tiene que correr mientras que haya algún coche vivo. Ese bucle tiene que hacer:
 - Mostrar en pantalla: **ESCRIBE NUMERO DE COCHE A MATAR**
 - Pedir un número por teclado
 - En base a ese número, buscarlo en la lista de coches, y al que tenga ese número, hacerle un:
 - Interrupt
 - Modificarle el valor de la variable “expulsado” a true.
- Al salir del bucle, decir: **TODOS LOS COCHES HAN SIDO EXPULSADOS**

CÓDIGO DE UTILIDAD:

Forma de sacar en 1 línea si alguno de los coches de la lista sigue vivo (teniendo la lista de coches como “hilosCoches”):

```
hilosCoches.stream().anyMatch(Coche::isAlive)
```